Lógica y funcionamiento del programa

Giselle Juliana Gómez, Joshua Castro Solarte

26 de octubre de 2026

1. Idea central

La temática del semestre es *Fantasías animadas de la historia de Colombia*. Nuestro videojuego reinterpreta, en clave de comedia animada, el denominado **Robo del siglo**, ocurrido en 1994 en el Banco de la República de Valledupar, donde una banda alcanzó la bóveda a través de un túnel excavado desde una casa cercana. En nuestra versión, los protagonistas son los **Minions**: Kevin y sus compañeros quieren conseguir botín para financiar una gran fiesta de bananas.

El juego comienza con una introducción animada, no jugable, que cuenta cómo los Minions prepararon un acceso subterráneo hasta el banco. Cuando empieza la partida, Kevin ya está dentro del edificio. El juego se divide en dos niveles con vistas y dinámicas claramente distintas:

- **Nivel 1, "La bóveda"**: infiltración en una vista lateral fija, sin *scroll*. Kevin recorre los pisos del banco, evita cámaras, láseres y objetos que caen, y debe dañar la puerta de la bóveda sin ser descubierto por un guardia centinela que se guía por el ruido.
- **Nivel 2, "La fuga"**: supervivencia en carro en una vista cenital desde arriba, con scroll vertical. El jugador controla el carro de fuga cargado con el botín y debe resistir hasta que termine el tiempo, evitando obstáculos en la vía. Cada impacto reduce la integridad del carro y puede hacer que se pierda parte del botín.

Aclaración sobre el tratamiento histórico: el juego no pretende narrar el hecho con exactitud; toma el túnel, la bóveda y la huida como elementos de inspiración y los lleva al terreno de la fantasía y el humor.

2. El protagonista y sus características

El protagonista es Kevin, un Minion controlado por el jugador. Tiene energía y vidas (tres vidas): pierde energía cuando toca un láser, lo golpea un objeto o choca con obstáculos, y pierde una vida cuando el policía lo atrapa. Tiene dos características de personalidad que cambian las decisiones del jugador:

1. **Bananero fortachón.** Al recoger una banana, Kevin entra en modo fortachón durante 10 segundos. *Beneficio:* daña la puerta de la bóveda el doble de rápido y puede empujar cajas ligeras que bloquean el paso. *Riesgo:* se mueve más lento y todo lo que hace produce un 50 % más de ruido.
2. **Payasín ruidoso.** El jugador puede activar a propósito una risa con una tecla. *Beneficio:* la risa genera un ruido fuerte en ese punto durante unos segundos, que atrae al policía hacia esa zona y lo aleja de la ruta del jugador. *Riesgo:* si el policía está cerca, encuentra a Kevin. Además, la risa tiene un tiempo de enfriamiento de 6 segundos, y el policía aprende: si cae varias veces en la distracción sin encontrar a nadie, deja de confiar en ella.

3. Nivel 1: "La bóveda"

3.1. Sinopsis del contexto

Kevin entra al banco usando el acceso subterráneo que prepararon los Minions. Dentro, un policía hace guardia y presta atención a cualquier ruido. Kevin debe llegar hasta la bóveda, dañar su puerta, recoger el botín mínimo y salir del banco sin ser atrapado.

3.2. Vista

Vista **lateral fija**, similar a una sección cortada del edificio. Se observan varios pisos conectados por escaleras, pasillos, cajas, estantes, cámaras de seguridad y la bóveda. Todo el banco cabe en una sola pantalla, por lo que no hay *scroll*.

3.3. Dinámica e interacciones

El jugador mueve a Kevin por los pisos del banco, salta y sube escaleras. Para dañar la puerta de la bóveda debe **mantener presionada una tecla**: una barra de resistencia de la puerta baja mientras la tecla esté presionada. El daño ya hecho se conserva si Kevin se interrumpe. Golpear la puerta produce un ruido continuo, así que el jugador debe decidir cuándo conviene golpear y cuándo detenerse.

Todo lo que hace Kevin produce ruido, y el policía se dirige hacia los ruidos que percibe. Las interacciones principales son:

- **Kevin – pisos y escaleras:** caminar, correr, saltar y cambiar de piso.
- **Kevin – puerta de la bóveda:** mantener la tecla para dañarla; el ruido dura mientras golpea.
- **Kevin – cámaras de seguridad:** cada cámara tiene un cono de visión que oscila de lado a lado. Si Kevin queda dentro del cono, se activa una alarma que genera un ruido muy fuerte.
- **Kevin – láseres:** tocar uno le resta energía y activa la alarma.
- **Kevin – objetos que caen:** algunos estantes son inestables. Si Kevin pasa corriendo cerca, sueltan un objeto que cae con trayectoria parabólica. Si lo golpea, pierde energía, y el impacto contra el piso produce ruido.
- **Kevin – cajas ligeras:** bloquean el paso. Solo se pueden empujar en modo fortachón y hacerlo produce ruido.
- **Kevin – botín:** una vez abierta la puerta, recoge lingotes y bolsas.
- **Kevin – banana:** al recogerla activa el modo fortachón durante 10 segundos.
- **Kevin – risa:** activarla genera un ruido fuerte en su posición durante 3 segundos.
- **Kevin – policía:** si el policía lo ve y lo alcanza, Kevin pierde una vida y vuelve a la entrada del banco. La puerta conserva el daño recibido.

3.4. Fuentes de ruido

Fuente	Tipo	Intensidad base
Caminar	`PASOS`	1
Correr	`CARRERA`	3
Golpear la puerta de la bóveda	`PUERTA`	5 (continuo mientras golpea)
Risa activada por el jugador	`RISA`	6 (dura 3 s)
Impacto de un objeto que cae	`IMPACTO`	4 (aumenta con la altura)
Empujar una caja ligera	`CAJA`	4
Tocar un láser o ser visto por una cámara	`ALARMA`	8

Tabla 1: Fuentes de ruido del Nivel 1. En modo fortachón la intensidad se multiplica por 1,5.

3.5. Objetivo

Dañar la puerta de la bóveda hasta abrirla, recoger el botín mínimo y llegar a la salida del banco sin perder las tres vidas.

3.6. Control por tiempo

Este nivel no tiene un temporizador general; la presión la ejerce el policía. Sí hay dos temporizadores de eventos: el modo fortachón (10 segundos) y el enfriamiento de la risa (6 segundos).

3.7. Retos y obstáculos

- Guardia centinela que reacciona al ruido y que aprende del comportamiento del jugador.
- Cámaras de seguridad con cono de visión oscilante.
- Láseres que obligan a saltar o a cambiar de ruta.
- Objetos que caen de estantes inestables.
- Cajas que bloquean el paso.
- Puerta con una barra de resistencia que hay que golpear haciendo ruido.

3.8. Físicas a implementar

1. **Movimiento armónico simple (oscilatorio).** El cono de visión de las cámaras de seguridad barre el pasillo con un ángulo que oscila:

   $$\theta(t) = \theta_0 + A\,\sin(\omega t + \varphi)$$

   Parámetros: ángulo central θ_0, amplitud A, frecuencia angular ω y fase φ. Se evidencia en el cono de luz que se mueve de lado a lado.

2. **Movimiento parabólico.** Se usa en el salto de Kevin y en los objetos que caen de los estantes:

   $$x(t) = x_0 + v_x\,t, \qquad y(t) = y_0 + v_{y0}\,t - \tfrac{1}{2}gt^2$$

   Parámetros: velocidad inicial, ángulo y gravedad g. En un objeto que cae desde un estante, $v_{y0} = 0$ y v_x es pequeña, de modo que cae con una trayectoria curva.

3.9. Agente inteligente: Guardia Centinela

En este nivel actúa el guardia controlado por la máquina que implementa el agente inteligente del proyecto. **Percibe** los ruidos del entorno (atenuados por la distancia) y a Kevin cuando entra en su campo de visión; **razona** si el ruido merece una investigación comparando su intensidad con lo que ha aprendido; **actúa** yendo hacia la fuente, persiguiendo o buscando; y **aprende** en qué zonas suele encontrar al jugador y qué tipos de ruido son señuelos. Su arquitectura se detalla en la sección 8.

3.10. Dificultad configurable

La dificultad no se limita a cambiar el tiempo. Al elegir fácil, normal o difícil se modifica el comportamiento del agente y de los peligros del entorno:

Parámetro	**Fácil**	**Normal**	**Difícil**
Radio de audición del policía d_0 (px)	150	220	300
Umbral de ruido para investigar θ	0,8	0,5	0,3
Tasa de aprendizaje α	0,10	0,25	0,50
Olvido del policía δ (1/s)	0,05	0,02	0,005
Frecuencia angular de las cámaras ω (rad/s)	1,0	1,6	2,4
Resistencia de la puerta (puntos)	100	150	220

Tabla 2: Valores iniciales de dificultad del Nivel 1 (se calibrarán en las pruebas). El daño base a la puerta es de 20 puntos por segundo, el doble en modo fortachón.

4. Nivel 2: "La fuga"

4.1. Sinopsis del contexto

Con el botín cargado en el carro de fuga, Kevin y sus compañeros salen del banco rumbo a su escondite. El camino está lleno de obstáculos y superficies peligrosas. Deben resistir hasta que termine el tiempo sin destruir el carro ni perder todo el botín..

4.2. Vista

Vista cenital con scroll vertical. La calle se observa desde arriba y se desplaza mientras el carro avanza. El jugador controla el carro de fuga dentro de los límites de la vía.

4.3. Dinámica e interacciones

El jugador controla el carro de fuga: puede acelerar, frenar y girar a izquierda o derecha. El carro transporta el botín obtenido en el banco. Las interacciones principales son:

- **Carro - conos y barreras:** el impacto reduce la integridad y hace perder una cantidad de botín.
- **Carro - huecos:** causan mayor daño y desvían temporalmente el carro.
- **Carro - charcos de aceite:** el carro derrapa, continúa deslizándose y pierde control parcial de la dirección durante el efecto.
- **Carro - botín:** si el carro choca repetidamente, puede perder lingotes o bolsas que llevaba.

- **Carro - límite de la vía:** salirse de la carretera reduce la velocidad y puede causar daño.

4.4. Objetivo

Sobrevivir hasta que el cronómetro llegue a cero, conservando integridad en el carro y al menos la cantidad mínima de botín requerida. Al cumplir ambas condiciones, el jugador gana el juego.

4.5. Control por tiempo

Este es el nivel controlado por tiempo. Tendrá un cronómetro de cuenta regresiva de 90 segundos en la configuración normal. El jugador gana cuando el tiempo llega a cero, siempre que el carro no haya perdido toda su integridad y conserve el botín mínimo.

4.6. Retos y obstáculos

- Conos de tránsito y barreras.
- Huecos en la vía.
- Charcos de aceite que producen derrape.
- Bordes de la carretera.
- Pérdida progresiva de integridad y botín después de varios impactos.

4.7. Físicas a implementar

1. **Fricción con desaceleración exponencial.** Al entrar en un charco de aceite, el carro se desliza y pierde velocidad:

$$v(t) = v_0\, e^{-\mu t}$$

   Parámetro: coeficiente de fricción μ, que depende del tipo de superficie (aceite, asfalto mojado, asfalto seco). Se evidencia en el deslizamiento del carro, Al entrar a un charco de aceite, el carro conserva parte de su velocidad y se desliza durante un tiempo. Mientras derrapa, responde menos a los giros del jugador.

4.8. Dificultad

La configuración principal de dificultad se aplica al Nivel 1. El Nivel 2 usa una duración fija de 90 segundos y una distribución definida de obstáculos, para mantener una implementación alcanzable. La dificultad real del nivel proviene de administrar la integridad del carro, evitar perder el botín y controlar el derrape.

5. Resumen de físicas por nivel

Nivel	Física	Elemento asociado
1	Oscilatorio (MAS)	Cono de visión de las cámaras de seguridad
1	Parabólico	Salto de Kevin y objetos que caen de los estantes
2	Fricción exponencial	Charcos de aceite

Tabla 3: Distribución de los modelos físicos del juego.

6. Sprites propuestos

Se utilizarán al menos cuatro sprites por nivel.

6.1. Nivel 1 (vista lateral)



Figura 1: Movimientos de los Minions

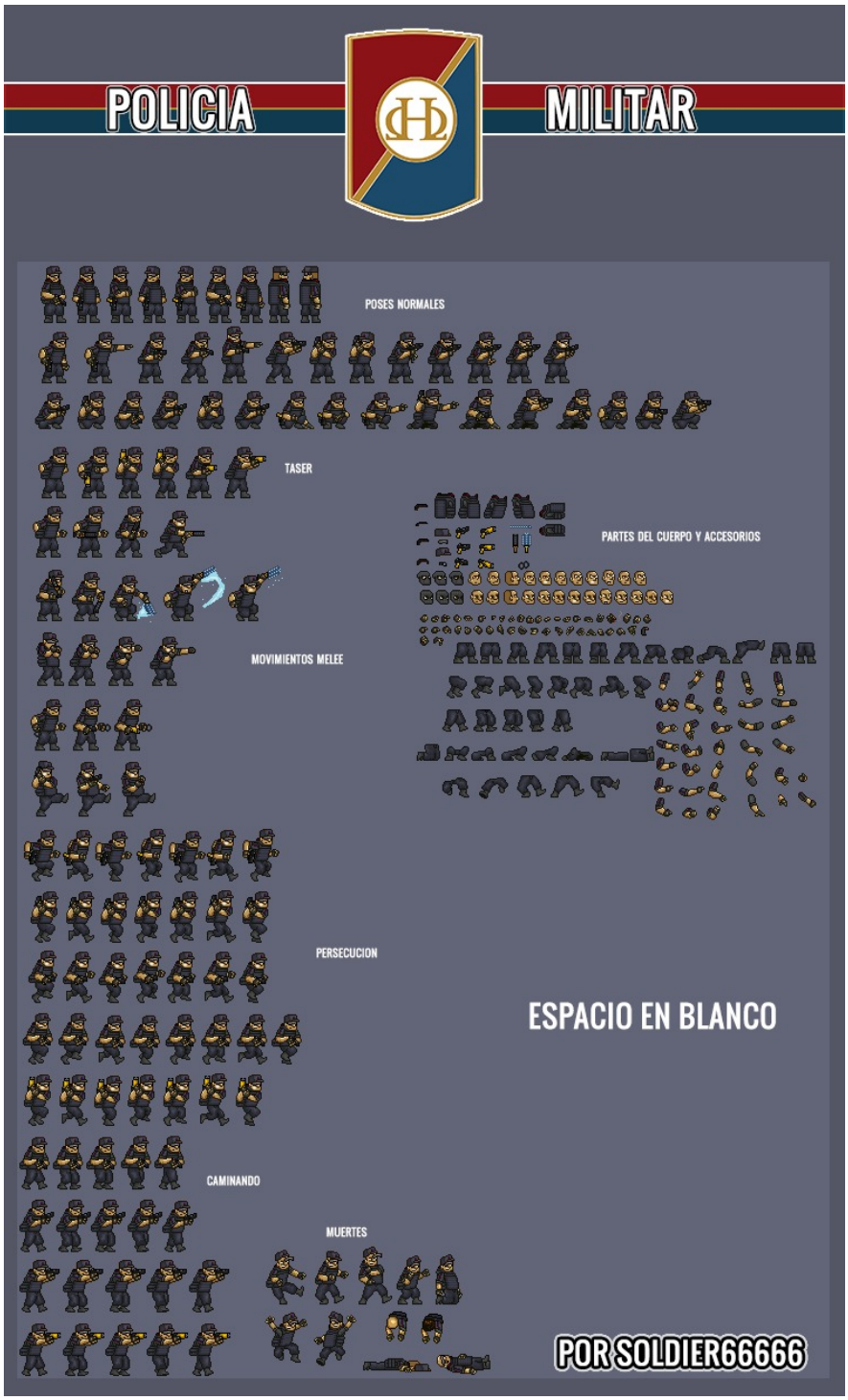


Figura 2: Policia



Figura 3: Bobeda cerrada, dañada y abierta



Figura 4: camara

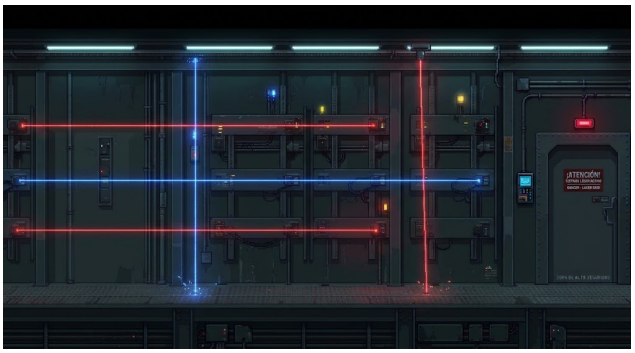


Figura 5: Lasers



Figura 6: lingotes de oro



Figura 7: Banana



Figura 8: Bolsa de robo

6.2. Nivel 2 (vista cenital)

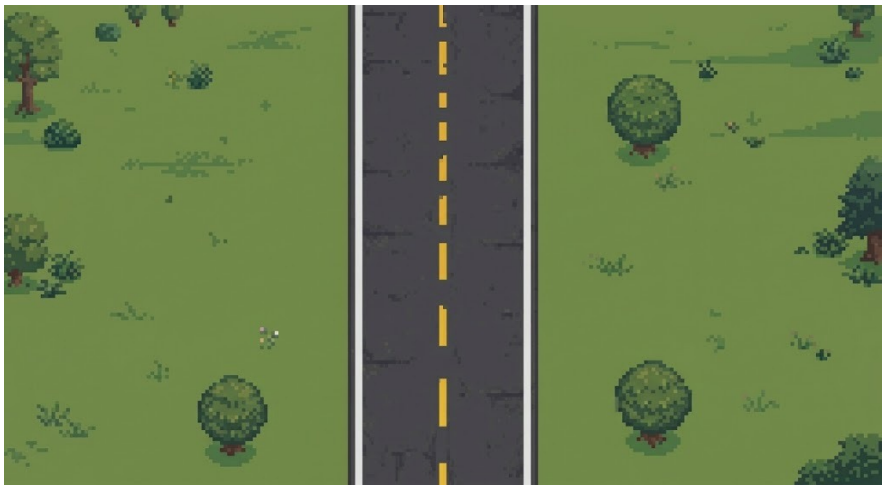


Figura 9: carretera

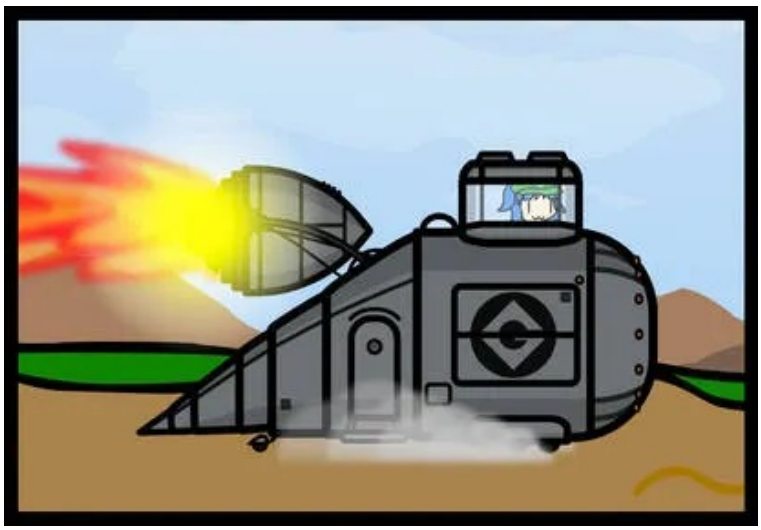


Figura 10: carro de fuga



Figura 11: Charco de aceite como obstaculo

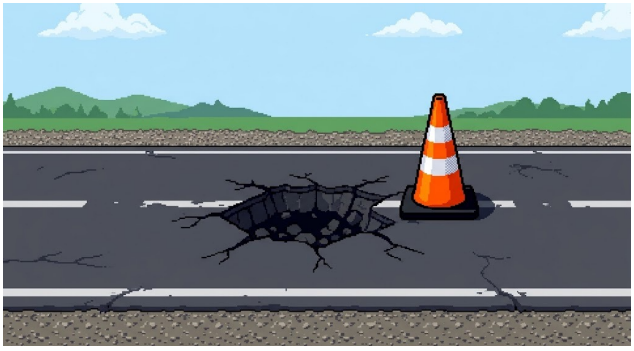


Figura 12: cono y hueco como obstaculos

7. Diagrama de clases de la capa lógica

El diseño separa la lógica del juego de la interfaz gráfica; en esta entrega se modela únicamente la capa lógica. Las jerarquías propias no heredan de Qt. Solo `Nivel` y `GestorJuego` heredan de `QObject`, porque necesitan señales y *slots*.

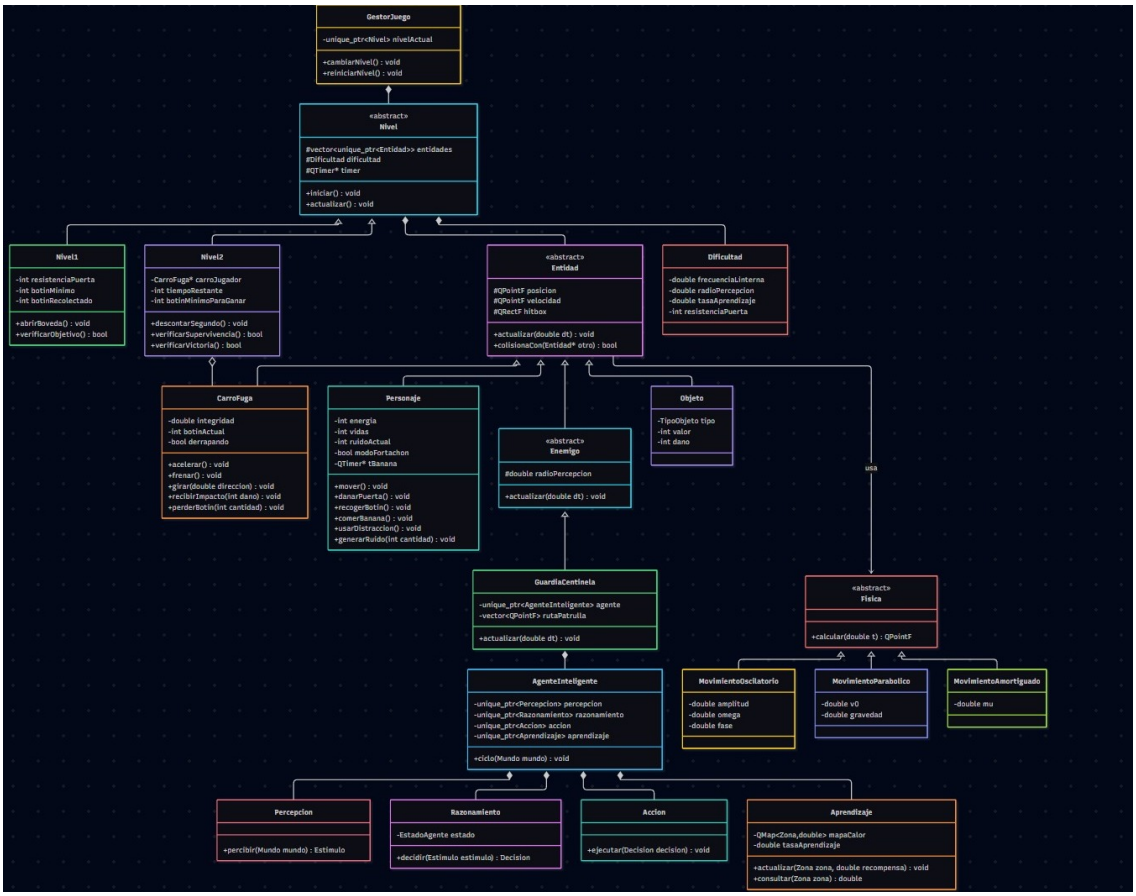


Figura 13: Diagrama de clases

7.1. Jerarquías de herencia propias

- **Entidad** (abstracta) → `Personaje`, `Enemigo`, `Objeto`, `PuertaBoveda`, `Obstaculo`, `Vehiculo`. Comparten posición, velocidad y *hitbox*, y obligan a implementar `actualizar(dt)`.
- **Enemigo** (abstracta) → `GuardiaCentinela`, que contiene al agente inteligente.
- **Obstaculo** → `CamaraSeguridad`, `ObjetoCaida`, `CharcoAceite`, `ConoBarrera`, `Hueco`.
- **Vehiculo** → `CarroFuga`.
- **Fisica** (abstracta) → `MovimientoOscilatorio`, `MovimientoParabolico`, `MovimientoAmortiguado`. Cada modelo es parametrizable y se asigna a una entidad mediante `calcular(t)`.
- **Nivel** (abstracta, hereda de `QObject`) → `NivelBoveda`, `NivelFuga`. Define el ciclo de vida de un nivel y emite las señales `jugadorPerdio` y `nivelCompletado`.

7.2. Relaciones principales

- **Composición:** `GuardiaCentinela` contiene un `AgenteInteligente`; el agente contiene sus cuatro componentes: percepción, razonamiento, acción y aprendizaje. `Nivel` contiene su objeto `Dificultad`.
- **Agregación:** `Nivel` agrupa las entidades activas mediante un contenedor dinámico; `Nivel2` administra el `CarroFuga`; `Entidad` puede usar un modelo de `Fisica`; y `GestorJuego` administra el nivel actual.

- **Dependencia:** `Percepcion` obtiene los estímulos del mundo; `Razonamiento` transforma un estímulo en una decisión; `Accion` ejecuta esa decisión; y `Aprendizaje` actualiza el mapa de calor según el resultado de la interacción.

7.3. Estructuras de datos por clase

Clase	Estructura	Uso
`Nivel`	`vector<unique_ptr<Entidad>>`	Entidades activas del nivel: personaje, guardia, carro, objetos y obstáculos.
`Aprendizaje`	`QMap<Zona, double>`	Mapa de calor con el nivel de sospecha de cada zona del banco.
`GuardiaCentinela`	`QVector<QPointF>`	Ruta ordenada de puntos para la patrulla del guardia.
`GestorJuego`	`unique_ptr<Nivel>`	Nivel actual administrado dinámicamente.

Tabla 4: Estructuras de datos principales

8. Arquitectura del agente inteligente: GuardiaCentinela

El Guardia Centinela es el agente autónomo del Nivel 1. Su ciclo se ejecuta en cada actualización del juego y separa explícitamente cuatro componentes, cada uno en su propia clase.

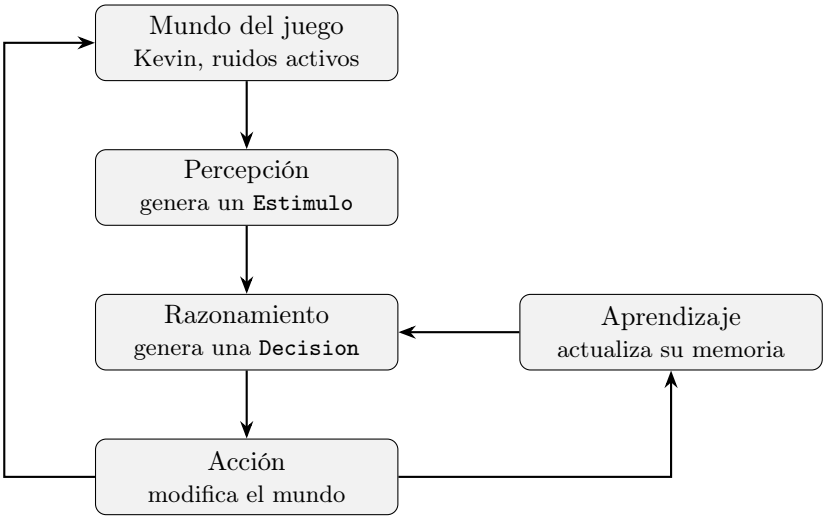


Figura 14: Ciclo del agente: percepción, razonamiento, acción y aprendizaje.

8.1. Percepción

El agente percibe dos tipos de estímulos. El primero es el **auditivo**: cada ruido del mundo llega atenuado según la distancia d entre el ruido y el policía:

$$I_p = \frac{I}{1 + (d/d_0)^2}$$

donde I es la intensidad base y d_0 el radio de audición. Si I_p no supera un mínimo, el ruido se ignora. El segundo es el **visual**: Kevin es visto si está dentro del campo de visión frontal del policía (un cono de $\pm 45°$ con alcance limitado). Cada detección se encapsula en un `Estimulo` con tipo de ruido, posición, intensidad percibida y una bandera de contacto visual.

8.2. Razonamiento

Combina una máquina de estados con una decisión basada en utilidad. Para interpretar un ruido, calcula:

$$U = I_p \cdot C_t \cdot (1 + Q_z) - \lambda\, d$$

donde C_t es la credibilidad aprendida del tipo de ruido, Q_z es la probabilidad aprendida de encontrar a Kevin en la zona del ruido y λ penaliza la distancia. Si U supera el umbral θ, decide investigar; si no, ignora el ruido. El contacto visual siempre dispara la persecución.

Estado	**Condición de transición**	**Siguiente**
`PATRULLAR`	$U > \theta$ para un ruido	`INVESTIGAR`
`PATRULLAR`, `INVESTIGAR`	Contacto visual con Kevin	`PERSEGUIR`
`INVESTIGAR`	Llega a la fuente y no hay nadie	`BUSCAR`
`PERSEGUIR`	Pierde de vista a Kevin	`BUSCAR`
`BUSCAR`	Termina el tiempo de búsqueda	`VOLVER`
`VOLVER`	Regresa a su ruta	`PATRULLAR`

Tabla 5: Transiciones de la máquina de estados del agente.

8.3. Acción

Ejecuta la decisión: camina hacia la fuente del ruido, persigue a Kevin, recorre el último lugar donde lo vio, o regresa a su ruta de patrulla. Al elegir el siguiente punto de patrulla, prefiere con mayor probabilidad las zonas con Q_z alto. Si alcanza a Kevin, este pierde una vida. Al terminar cada acción devuelve un `Resultado` con la zona, el tipo de ruido y la recompensa r.

8.4. Aprendizaje

El agente mantiene dos memorias: un **mapa de calor** Q_z por zona del banco (qué tan probable es encontrar al jugador ahí) y una **credibilidad** C_t por tipo de ruido (qué tan confiable es ese ruido como señal de Kevin). Tras cada interacción las actualiza con

$$Q_z \leftarrow Q_z + \alpha\,(r - Q_z), \qquad C_t \leftarrow C_t + \alpha\,(r - C_t)$$

donde α es la tasa de aprendizaje (ligada a la dificultad), $r = 1$ si encontró a Kevin y $r = 0$ si fue una falsa alarma (por ejemplo, una risa de distracción). Q_z empieza en 0 y C_t en 1. Con el tiempo las zonas se van olvidando: $Q_z \leftarrow Q_z(1 - \delta\,\Delta t)$. El resultado es que el policía patrulla más donde el jugador suele hacer ruido y deja de caer en la risa si se repite sin que nadie aparezca, por lo que el jugador debe variar sus estrategias. La memoria del agente se reinicia cada vez que se crea el nivel: al empezar una partida nueva o al reiniciar el nivel, el guardia vuelve a $Q_z = 0$ y $C_t = 1$. Dentro de un mismo intento sí se conserva, incluso si Kevin pierde una vida y vuelve a la entrada.

9. Preguntas esenciales

9.1. Contenedores seleccionados y justificación

- `std::vector<std::unique_ptr<Entidad>`: guarda las entidades activas de cada nivel. Es contiguo en memoria y rápido de recorrer en cada *frame*; los `unique_ptr` gestionan la memoria dinámica y permiten polimorfismo con la clase base abstracta. Las entidades se crean dinámicamente porque pueden aparecer o desaparecer durante la partida, por ejemplo un objeto que cae en el banco o botín que se pierde después de un choque del carro. Se descartó `std::list` porque no se insertan elementos en el medio y se prioriza el recorrido secuencial rápido.

  `QMap<Zona,double>`: representa el mapa de calor del Guardia Centinela. Cada zona del banco se asocia con un valor de sospecha que aumenta cuando Kevin genera ruido allí con frecuencia. Se usa `QMap` porque permite consultar una zona por clave y solo guarda las zonas relevantes; para usarlo, `Zona` implementa `operator<`.

  `QVector<QPointF>`: guarda la ruta de patrulla del Guardia Centinela. Es una secuencia ordenada de puntos con acceso por índice, adecuada para recorrer los pasillos del banco.

  `std::unique_ptr<Nivel>`: en `GestorJuego`, administra el nivel actual. Garantiza que el nivel se destruya automáticamente al cambiar o reiniciar la partida, evitando fugas de memoria.

9.2. Uso de excepciones

Se definen excepciones propias que heredan de `std::runtime_error`:

- `ExcepcionRecurso`: se lanza si un sprite o sonido no se puede cargar.
- `ExcepcionNivel`: se lanza si un nivel tiene una configuración inválida (por ejemplo, sin puerta, sin salida o sin botín mínimo).

Además, los modelos físicos lanzan `std::invalid_argument` si reciben parámetros imposibles (por ejemplo, un coeficiente de fricción menor o igual que cero), y se captura `std::out_of_range` al acceder a elementos fuera de un contenedor. Se usan bloques `try/catch` en la carga de recursos y en `GestorJuego::cambiarNivel`, para que un recurso faltante muestre un mensaje claro en lugar de cerrar el programa de forma abrupta.

```
class ExcepcionRecurso : public std::runtime_error {
public:
    explicit ExcepcionRecurso(const std::string& msg)
        : std::runtime_error(msg) {}
};

MovimientoAmortiguado::MovimientoAmortiguado(double m) : mu(m) {
    if (m <= 0.0)
        throw std::invalid_argument("mu debe ser positivo");
}

try {
    cargarSprite("policia.png");
} catch (const ExcepcionRecurso& ex) {
    qWarning() << ex.what();
}
```

9.3. Uso de conexiones (señales y slots) y sintaxis

Las conexiones de Qt permiten que un objeto emisor avise a un receptor sin que las clases se conozcan directamente, lo que reduce el acoplamiento. La sintaxis es `connect(emisor, &Clase::senal, receptor, &Clase::slot)`.

```
connect(timer, &QTimer::timeout, this, &NivelBoveda::actualizar);
connect(cronometro, &QTimer::timeout,
        this, &NivelFuga::descontarSegundo);
connect(nivel, &Nivel::jugadorPerdio,
        gestor, &GestorJuego::reiniciarNivel);
connect(nivel, &Nivel::nivelCompletado,
        gestor, &GestorJuego::cambiarNivel);
```

El temporizador del nivel dispara `actualizar()` en cada ciclo; el cronómetro del Nivel 2 descuenta un segundo en cada tick; y el nivel avisa al gestor cuando el jugador pierde o completa el nivel.

10. Cambios respecto al Momento I

En el Momento I únicamente definimos la temática general del juego: los Minions y el Robo del siglo de Valledupar (1994). En esa primera versión la idea era distinta: los Minions eran **detectives** que investigaban el caso y buscaban resolver el robo. Al analizar la dinámica que podíamos construir, decidimos que los Minions sean ahora **los ladrones**. Nos gustó más porque se presta mejor a un videojuego de acción y de ingenio, con dos niveles muy diferentes entre sí: una infiltración sigilosa guiada por el ruido y una persecución en carro.

Aspecto	**Momento I (idea inicial)**	**Momento II (versión actual)**
Rol de los Minions	Detectives que resuelven el caso	Ladrones que ejecutan el golpe
Nivel 1	Sin definir	"La bóveda": vista lateral fija, puerta que se daña con una tecla y policía que sigue el ruido
Nivel 2	Sin definir	"La fuga": conducción cenital en carro, con cronómetro, obstáculos fijos, pérdida de integridad y pérdida de botín por impactos.
Agente inteligente	Solo la idea general	Guardia centinela con percepción, razonamiento, acción y aprendizaje
Físicas	Sin definir	Tres modelos: oscilatorio, parabólico y fricción

Tabla 6: Comparación entre la propuesta del Momento I y la del Momento II.

Además, en el Momento II concretamos los detalles que en el Momento I solo estaban planteados como idea: las vistas y dinámicas de los niveles, las características del protagonista (bananero fortachón y payasín ruidoso), los parámetros de cada modelo físico, la configuración de dificultad, el diseño del agente inteligente y el diagrama de clases de la capa lógica.